

AI Trust: Practice Theory — A Worked Example (Calendar Stewardship)

Monyet Batu

ORCID: 0009-0007-9002-5381

27 May 2026

A follow-up to [AI Trust and Situated Awareness: A Practice Theory Reframe](#), [Practice Theory — The Apprenticeship and a Strange Loop](#), and [Practice Theory — The Implementation](#).

Reader's note: the first three essays in this series made the conceptual case, named the strange loop, and built the substrate. This essay does something narrower. It picks one concrete failure that connected agents are running into right now, and shows what a single small practice looks like as the answer. The artifact is in the companion repository; the trail it produces is on the page. The worked example is Calendar Stewardship; the failure mode is the one most readers have already felt.

A failure most readers have felt

A common pattern with calendar-connected agents: ask it to move a meeting an hour later, watch it move the meeting. On the surface that looks like changing a time and clicking save. Underneath, the action notifies the attendees in real time, moves the prep block the user set aside the day before, breaks a commitment one of those attendees had made to a customer, and turns a private conversation into a meeting that now conflicts with something more important on the other end. The agent did exactly what was asked. The damage is what the agent didn't understand it was doing.

This is the failure mode the first essay in this series called *access without situated awareness*. The agent had access to the calendar API. It did not have a practitioner's working sense of where it was in the work, what the goal was, what was at hand, or what would have been a legitimate next step. It had no practice — no teleo-affective, no understanding of what a meeting *is* to the people on it, no rules about staging before issuing, no affordances naming the user's stance as a thing to be invited rather than assumed. Without those, every calendar action is one step long: edit the event, send the invites, done.

The three prior essays argued that the missing layer is **situated awareness**, that it is delivered by a practice bundle, that the bundle is transmitted through apprenticeship, and that the whole arrangement is kept honest by an autonomic loop. This essay narrows from the architecture to *one practice* — Calendar Stewardship — and walks through what changes when its situated awareness is projected against exactly this scenario.

What's missing in one sentence

Access without situated awareness. The agent can call `update_event(send_updates='all')`. It cannot tell, from the API alone, where it is in the work, that the action is irreversible at the messaging layer, that it puts notifications in three external inboxes, that it changes a commitment between people. A bundle is the captured form of the situated awareness a practitioner brings to this work — the **teleo-affective** (the goal-with-orientation: where the practice is going and what posture it goes there with), the **understanding** (the perceptual frame: how to read what's in front of you in this practice's terms), the **rules** (the bright lines

the situation must respect), the **affordances** (what is available to be done here, in the practice's own vocabulary), and the **materials** (the executables behind them). Not a plan with milestones. A practitioner's working knowledge of the practice, captured. The rest of this essay shows the bundle. Then it shows the trail.

One small practice: Calendar Stewardship

Bundle: Calendar Stewardship

id : calendar_stewardship

mode : somatic

description : Tend the user's calendar as a record of commitments to people – not as fields to be edited. Stage proposed

changes;

invite the user's stance before any change that notifies attendees; issue deliberately, never silently.

teleo_affective:

Steward of the user's time and commitments – Tend the user's calendar as a record of commitments to people, not as fields to be edited. A meeting on the calendar is something the user agreed to with other people who have arranged their day around it. Moving it has consequences for them. Be the practitioner who makes those

consequences

visible to the user before acting, not the one who acts and tells them after.

understanding:

A meeting is a commitment, not a slot – A calendar event is not a row in a database; it is a commitment made to the people listed as attendees. When an attendee is external (outside the user's organisation), rescheduling means a notification lands in their inbox, their day shifts, and the user's relationship with them shifts too. The mechanical act of editing the event is trivial; the relational act of moving the meeting is not. Two surfaces exist for any change: staging (no one is notified yet; the change is a proposal for the user to review) and issuing (the change is on the wire and cannot be unsent). Treat them as different things – because they are.

rules:

- Stage before you issue – Never issue a calendar change without a prior staging on the same event. Staging is the review window; issuing without it bypasses the user.
- Invite the user's stance before issuing – Before issuing any reschedule of an event with external attendees, name the choice and hand back. Do not issue past a choice the user has not made.
- No silent attendee changes – If a change affects attendees, the issuance must use `send_updates='all'`. Suppressing notifications on a change attendees would feel is a violation, even if technically possible.

affordances:

- `read_calendar` – List upcoming events in a date range; see what is

- there before proposing any change. Surfaces attendee counts and an external-attendee flag.
- `propose_reschedule` – Stage a reschedule on an event. No attendees notified; no invite changes on the wire. Always the first step.
 - `invite_stance` – Name the choice that belongs to the user and hand back. Required between propose and issue for external-attendee changes.
 - `issue_reschedule` – Convert a staged reschedule into an issued change. Notifications go to every attendee. Irreversible at the messaging layer. Requires a prior propose and (for external attendees) a prior `invite_stance`.

materials:

- `cal_list_events`, `cal_propose_reschedule`, `cal_invite_stance`, `cal_issue_reschedule`
(Google-Calendar-shaped mock; side effects printed as [CALENDAR MOCK] WOULD NOTIFY: ... rather than actually sent.)

A few things to notice in this captured form, none of them about the calendar.

The **teleo-affective** does not describe the API. It describes a stance: steward, not editor; the practitioner makes consequences visible before acting. An LLM engaging this bundle is being cued into that posture before any function call is mentioned.

The **understanding** teaches the LLM the *two surfaces* that matter for any change — staging and issuing — and names the asymmetry between them. *The mechanical act is trivial; the relational act is not.* This is the kind of thing an apprentice would absorb through years of watching a calendar-running executive assistant work. Here it has to be written, once.

The **rules** are three bright lines. Each is short. Each is independently checkable from the trail.

The **affordances** are not a one-to-one mapping over API methods. There is no `update_event` affordance, because *update_event is not a thing in this practice* — the practice has split it into a proposal and an issuance, with an explicit pause in between. Same single underlying API call; very different framings.

The **materials** are the executables. The mock is shaped like the real Google Calendar API; the side effects are *printed* with a [CALENDAR MOCK] prefix instead of actually leaving the process. The print is the demonstration: the saved harm made visible.

The same calendar move, with the bundle projected

The companion repo's verify exercises Calendar Stewardship end-to-end. The relevant fragment of the trail it produces:

```
enactment <id> bundle=calendar_stewardship parent=<engagement-enactment>
  [4] read_calendar / cal_list_events
      arguments : {start_date: "2026-05-27", end_date: "2026-06-03"}
      result    : [{id: "evt-customer-review", attendee_count: 3,
                    has_external_attendees: true}, ...]
```

```
[5] propose_reschedule / cal_propose_reschedule
    arguments  : {event_id: "evt-customer-review",
                  new_start: "2026-05-29T15:00:00+00:00",
                  new_end:   "2026-05-29T16:00:00+00:00",
                  reason:     "User has a conflict at the original time;
                              moving an hour later in the same day."}
    result     : {staging_id: "stg-...", send_updates: "none",
                  notified: []}

[6] invite_stance / cal_invite_stance
    arguments  : {question: "Acme customer review has external attendees
                              (alice@acme.example, bob@acme.example).
                              Issue the reschedule and notify them?",
                  options:   ["Issue with notifications", "Hold",
                              "Cancel reschedule"]}
    result     : {id: "stance-...", asked_at: "..."}

[7] issue_reschedule / cal_issue_reschedule
    arguments  : {staging_id: "stg-..."}
    result     : {send_updates: "all",
                  notified: ["alice@acme.example",
                              "bob@acme.example",
                              "carol@us.example"]}
```

And what the mock prints alongside those steps:

```
[CALENDAR MOCK] STAGED reschedule of 'Customer review with Acme'
                  ...send_updates='none' (0 attendees notified)
[CALENDAR MOCK] STANCE REQUESTED: 'Acme customer review has external
attendees (alice@acme.example, bob@acme.example). Issue
the reschedule and notify them?'; options=[...]
[CALENDAR MOCK] ISSUED reschedule of 'Customer review with Acme';
send_updates='all'; WOULD NOTIFY:
['alice@acme.example', 'bob@acme.example',
 'carol@us.example']
```

Four steps, in that order. The *0 attendees notified* at step 5 is the staging window. The recorded question at step 6 is *the user's voice being awaited* — not assumed. The three names at step 7 are not what was sent; the mock saved them from being sent. They are *what would have left the process if this were the real API binding, after the user authorised it*. The trail makes both halves visible.

Contrast with the bare-LLM path. The bare LLM, given only the raw Google Calendar API, calls `update_event(eventId, send_updates='all')` once. The trail records one step. Three notifications fly. The user finds out when their phone buzzes thirty seconds later and one of the external attendees has already replied with *"this conflicts with my other meeting."*

The discipline is the difference between one step and four. And you can read it off the trail.

What the Judge would catch if it slipped

The strange loop named in essay 2 and built in essay 3 isn't only for the architecture's own bundles. It applies here too.

Suppose a future enactment of Calendar Stewardship cuts a corner — calls `issue_reschedule` directly, without a prior `propose_reschedule`. Two things happen.

The mock errors out. `cal_issue_reschedule` requires a `staging_id`, and there isn't one. The step fails. The trail records the failure. Nothing was notified.

A Judge enactment, reading the trail later, names the violation as Friction. The Judge bundle's understanding describes a `rule_neglect` kind for exactly this case: a rule is in the practice, the trail shows it was not honoured, the Judge emits an observation with the evidence (the rule's id, the missing step, the enactment it would have belonged to). A Smoother enactment then has the option of amending the bundle's content — adding a rule that names a more specific safeguard, sharpening an affordance description that turned out to be too easy to misread, even amending the material so its error message tells future enactments where the staging step would have gone.

One part of the discipline is enforced mechanically at the **data layer** (`cal_issue_reschedule` will not accept a free-standing issuance without a staging id), and the rest is held at the **bundle layer** — the three rules constrain the enactment shape an LLM enacting the bundle will follow, and the trail records the steps so the discipline is inspectable. The Judge can read the trail and name violations of rules that aren't mechanically enforced; the Smoother can amend the bundle (or harden the material) in response. So: one rule is enforced in code, all three are inspectable from the trail, and the gap between *inspectable* and *enforced* is exactly the space the autonomic loop is built to close. *Trust as enacted structure*, applied to a single calendar move.

Going up a level: a Personal Secretary

Calendar Stewardship is small on purpose. The architecture's scaling story lives in *composition* — practices that compose the affordances of other practices, the way an experienced executive assistant holds calendar work and email work as one continuous attendance.

A Personal Secretary bundle would have its own teleo-affective ("attend to the user's correspondence-and-time as one thing"), its own understanding (the email and the calendar are two surfaces of the same standing relationships), its own rules (no commitment on the user's behalf; drafts are offerings; staging applies to both invites and replies). Its affordances would include `propose_reschedule` and `issue_reschedule` from Calendar Stewardship, plus draft-not-send affordances for email, plus an `invite_stance` that spans both surfaces, plus an `ultra_vires` affordance that lets the practice declare its own limit and hand back when a step would require more authority than the bundle carries.

A Correspondent practice along these lines exists in a separate `practice-projection` repository the author maintains, exercising the same composing shape against email and calendar surfaces. That artifact is not part of the inspectable evidence chain of this paper — readers wanting to confirm a working real-API binding will need to wait for it to be published with its own verify and trail. What this paper *does* show is that the bundle shape, the projection rules, the trail, and the Judge are unchanged across domains: extending Calendar Stewardship into a Personal Secretary that holds calendar + email + procurement + customer-success-handoff means selecting more from the same pools, not redesigning the substrate. Scaling is *cheap* in the architectural sense; whether any particular composing practice has been built and exercised against real production APIs is a separate question and is not claimed here.

A note on the mock and the swap to real

The materials shown here print rather than send. That choice is structural, not provisional. The bundle's *capture* (description, schema, framing) is independent of the *executable* behind each material's name — Step 1 of the implementation essay names this separation explicitly. Swap `cal_issue_reschedule`'s callable for one that hits the real Google Calendar API and the bundle, the projection, the trail, and the Judge all keep working unchanged. The only difference is that the **WOULD NOTIFY** print becomes an actual notification.

The print form has one virtue the real binding doesn't have: **it makes the failure mode reproducible without harm**. The case study can show what would have happened in the bare-LLM run without sending three real emails to people who didn't ask to receive them. That's a useful property for a published worked example. The architecture supports both; the demonstration uses the safer form.

The lineage

The frame this essay turns on — *situated awareness, not a plan* — is Lucy Suchman's, from her 1987 *Plans and Situated Actions*. Plans, she argued, do not *cause* action; they are a *resource* people use while acting in situ. The bundle is exactly that resource, in capturable form. The Calendar Stewardship bundle is the resource. The customer review with Acme is the situation. The four-step enactment is the *situated action* through which the resource meets the world. The agent that fails the calendar-move test does so because it has been handed a plan — *do A, then B* — without the situated awareness that would let it know where the move actually sits.

The other figure behind the architecture is **Donald Schön**, whose 1983 *The Reflective Practitioner* studied how professionals actually think — *knowing-in-action* (the tacit competence a practitioner has while doing the work) and *reflection-in-action* (stepping back mid-doing to examine and adjust). The bundle's *understanding* is Schön's knowing-in-action made explicit and transmissible; the Judge and Smoother are reflection-in-action at the system scale; the trail is the substrate that makes reflection possible at all. Schön needed *something to reflect on*; prose-only systems give the practitioner nothing. The trail is what.

The case this essay makes sits inside a forty-year tradition. Schatzki named practice as the unit of ontological analysis; Schön named the practitioner's reflective competence; Suchman named the situated character of action against any plan. The architecture in the companion repo is what happens when those three are taken seriously for the LLM case.

What this essay claims, and what it doesn't

It claims one thing: **the situated awareness the calendar-move failure points at is something a small practice bundle can carry, today, against a Google-Calendar-shaped surface, with the discipline inspectable from the trail and a real binding swappable behind the material name**. The bundle in this essay is forty lines of captured content. The mock is a hundred lines of Python. The verify is twenty lines of MCP tool calls. The whole worked example is small.

It does not claim that bundles are sufficient for every agent failure. It does not claim that the autonomic loop converges in production without further work — essay 3's Step 12 names that gap. And it does not claim Calendar Stewardship is a finished design. The bundle as shown stewards the move *against the user* (staging, then inviting the user's stance) but not yet *against the attendees* — once the user authorises, the reschedule is issued and the attendees first hear about it as a notification, with no separate consent step. A

more careful version would split `issue_reschedule` further: a `propose_to_attendees` affordance that sends a low-friction proposal first, an `await_responses` holding affordance, and the actual issuance only after responses come in (or an explicit user override with a recorded reason). That is exactly the kind of gap a Judge enactment, reading repeated enactments of this bundle in the trail, would name as `rule_neglect` against the teleo-affective's "make consequences visible before acting" — and the kind a Smoother would close through `pm_amend_bundle`. The same machinery that holds the bundle is what would extend it. A real deployment would also pick up rules around cross-time-zone moves, recurrence handling, conflict detection, and a dozen other things a working EA holds in their head.

A worked-example aside before the close. Three pieces of evidence emerged from running this case study against a real LLM harness — Codex driving the somatic MCP server, with a natural-language prompt and no scaffolding. The full session is captured in `case-study-codex-transcript.md`; the short version follows.

The mechanism works under a real LLM. Codex switched into Calendar Stewardship, read the calendar, staged the reschedule with no notifications, and stopped to invoke `invite_stance` before issuing — the same four-step shape the verify exercises deterministically, now produced by a real LLM in production-equivalent conditions. The bundle's discipline held.

The architecture surfaced its own gap. The first run tried to call the calendar's listing material with the parameter names `start` and `end`; the material's schema declares `start_date` and `end_date`. The reason the guess was needed: `discover_affordances` returned material *names* but not their *schemas*, so the harness LLM saw the affordance existed but had to confabulate the argument shape on first try. The fix — surface each material's `input_schema` inline in the `discover_affordances` result — is the kind of small architectural correction the trail-as-evidence loop is built to make legible. One round of use closed a gap the verify had not shown.

The LLM under the architecture reasons in the practice's register. Asked to explain its work, Codex paraphrased the bundle's three rules in its own words (*"read the calendar first, stage before issuing, and do not silently change meetings with external attendees"*) and articulated its reasoning from the bundle's understanding to the action: *"the somatic calendar practice treats a meeting with external attendees as a commitment involving other people. So I could safely stage the change, but I should not issue it and notify attendees until you confirm."* That is Suchman's situated action with an LLM as the actor — the bundle was not a script the LLM followed; it was a resource the LLM held while acting in the situation. The understanding (*"a meeting is a commitment, not a slot"*) reached the LLM's reasoning and produced an action it could defend on the bundle's own terms.

Practices improve through use; the architecture makes the improvement legible; and the LLM under that architecture reasons in the practice's register rather than the API's. The same loop story applies to the bundle's own missing-attendee-consent step — a gap the architecture is already shaped to surface and close when it next runs.

What it claims is that the *shape of the answer* is in your hands. Essay 1 of this series named the missing layer as situated awareness and argued that its carrier is a practice. Essay 2 argued that the practice is transmitted through apprenticeship. Essay 3 built the substrate. This essay narrows to a single practice and shows what the answer looks like at the smallest scale that still makes a meaningful claim.

The bundle is the unit of situated awareness. The trail is the unit of trust. Both are at HEAD in the repository — runnable with one command.

Series

This is the fourth of four essays in the AI Trust series:

1. *AI Trust and Situated Awareness: A Practice Theory Reframe* — DOI.
2. *Practice Theory — The Apprenticeship and a Strange Loop* — DOI.
3. *Practice Theory — The Implementation* — DOI pending Zenodo deposit.
4. **This essay.** *Practice Theory — A Worked Example (Calendar Stewardship)* — DOI pending Zenodo deposit.

Companion software: [practice-theory-implementation](https://github.com/HiddenDeveloper/practice-theory-implementation) — DOI pending Zenodo deposit. Source at <https://github.com/HiddenDeveloper/practice-theory-implementation>.